Adaptive Concept Drift-Aware Intrusion Detection System for IoT Networks

Mahesh Gutta
Department of Computer Science
University of Galway, Ireland
m.gutta1@universityofgalway.ie

***Abstract*—Static machine-learning intrusion detection systems (IDS) for the Internet of Things (IoT) are trained once and never updated, yet IoT traffic drifts continuously as firmware changes, new attack variants emerge, and usage patterns shift. This paper presents an Adaptive Concept Drift-Aware IDS (ACD-IDS) built around a Random Forest classifier monitored by the ADWIN drift detector, which triggers retraining only when a statistically significant shift in prediction error is confirmed, and which is protected from catastrophic forgetting by a reservoir-sampling rehearsal buffer. The system is evaluated on CICIoT2023 (34 classes: benign plus 33 attack types) under both a static-split baseline and a synthetic three-phase drift stream. Adaptation raises mean streaming accuracy from 45.09% to 77.26% (+32.17 percentage points) and multi-class macro F1 from 0.191 to 0.473, at a total retraining cost of 0.21 seconds across four retrains. The rehearsal buffer recovers Phase-1 held-out macro F1 from ∼0.01 (uncontrolled forgetting) to 0.67–0.69 once tuned. ADWIN is further benchmarked against hand-coded DDM and EDDM detectors: all three land within 0.05 macro F1 of each other on final accuracy, but differ meaningfully in reaction latency, with ADWIN retraining within three chunks of both simulated phase transitions. The system is positioned against the project's base paper, Liu (2025) – a static three-layer IoT IDS with no drift handling – and against its closest methodological comparator, Abdel Wahab (2022), an online deep-learning approach that updates continuously and explicitly does not attempt to prevent forgetting. Full precision/recall/F1/accuracy reporting and confusion-matrix analysis are provided throughout.**

***Index Terms*—Intrusion Detection, IoT, Concept Drift, ADWIN, DDM, EDDM, Random Forest, Catastrophic Forgetting, Rehearsal Buffer, CICIoT2023.**

I. Introduction

THE Internet of Things now spans billions of devices across healthcare, manufacturing, smart homes, and critical infrastructure, and this growth has opened up an attack surface that keeps expanding. Machine-learning-based intrusion detection systems routinely report accuracy above 99% on public benchmarks, but those figures come from a fixed test set drawn from the same distribution as the training data. Real deployments do not sit still: firmware gets updated, new attack variants show up, and usage patterns shift with time. This is *concept drift* – the statistical relationship between traffic features and attack labels changes over time – and it wears a frozen model down quietly, with no built-in alarm, until it simply stops catching real intrusions.

This paper builds and evaluates an IDS that detects concept drift on its own, using the ADWIN algorithm [1], and only retrains once drift is confirmed, while a long-term rehearsal buffer protects previously learned attack patterns from being forgotten. The project's base paper is Liu (2025) [6], a static three-layer hierarchical IoT IDS from the same research group. This project borrows the general IoT-intrusion-detection framing from Liu but not the three-layer architecture, since Liu's thesis never touches on concept drift, streaming evaluation, or adaptive retraining anywhere. The closest methodological comparator found in the literature is Abdel Wahab (2022) [7], which tackles concept and data drift in IoT IDS head-on, but through a very different and considerably more expensive mechanism, discussed in detail in Sections II and VII.

Contributions. The contributions of this paper are fivefold:

1) **A drift-adaptive layer the base paper doesn't have.** Liu (2025) never touches concept drift, so adding automatic drift detection and triggered retraining on top of the same IoT-IDS problem is genuine novelty here, not something the thesis already flagged as future work.
2) **A catastrophic-forgetting fix, tuned by hand.** An early version of the system that cleared its buffer on every retrain saw Phase-1 held-out macro F1 collapse to ∼0.01. A reservoir-sampling rehearsal buffer, tuned through a documented capacity/rehearsal-count sweep, brings this back up to 0.67–0.69 while still adapting strongly to new phases.
3) **A three-detector comparison that goes beyond aggregate F1.** ADWIN, DDM, and EDDM are compared not just on final accuracy but on how quickly each one reacts (chunks-to-retrain after a phase boundary), something the closest comparable published comparison [13] does not report. A bug in the hand-coded DDM/EDDM implementations – an internal `reset()` call that silently wiped the drift flag before it could be read – was caught and fixed during this work.
4) **Full multi-class evaluation under drift.** The system is tested against the complete 33-attack-type CICIoT2023 taxonomy under a genuine temporal three-phase drift stream, with leak-free held-out generalisation testing per phase – a harder and more realistic setup than the binary, cross-validated evaluations used in most of the reviewed prior work, Abdel Wahab (2022) included.
5) **A quantitative and capability comparison against both the base paper and the closest related work.** Common metrics (accuracy, macro F1) are placed side by side with Liu (2025), with clear caveats about differing datasets

and architectures, alongside a capability table showing what each system does and doesn't measure. A detailed technical contrast against Abdel Wahab (2022) rounds this out.

The rest of this paper is organised as follows. Section II reviews related work and identifies the gap. Section III describes the dataset and notation. Section IV defines the detection and adaptation methods. Section V covers the experimental setup. Section VI reports results, including full precision/recall/F1/accuracy tables and confusion matrices. Section VII discusses the findings against the base paper, the closest related work, and the wider literature, and names threats to validity. Section VIII concludes.

II. Related Work

Ten papers were selected through a structured search across IEEE Xplore, ScienceDirect, and Semantic Scholar, using combinations of "intrusion detection", "IoT", "concept drift", "ADWIN", and "adaptive learning". Inclusion required peer-reviewed status, publication between 2019–2026 (except the foundational ADWIN paper), and direct relevance to IoT IDS, concept drift, or adaptive learning for network security.

A. The base paper: a static hierarchical IoT IDS

Liu (2025) proposed a three-layer hierarchical IDS: an autoencoder for normal/attack filtering, XGBoost for known-vs-unknown gating, and an RNN-BiLSTM for fine-grained classification, evaluated on Bot-IoT and two ToN-IoT variants [6]. It reports 92–95% accuracy along with edge-deployment metrics ($\sim$7.3–7.4 ms/sample latency, $\sim$135 samples/sec throughput, under 450 MB memory, under 3.1% CPU). The architecture handles IoT traffic heterogeneity well, but like most IDS work it assumes a static model that is never updated. Liu's own results also point to per-module weaknesses worth noting here: the RNN-BiLSTM stage struggles with payload attacks such as Injection and XSS, which happens to be this project's hardest evaluation phase (Phase 3, Web Attacks).

B. ADWIN and alternative drift detectors

ADWIN (Bifet & Gavaldà, 2007) keeps a variable-length sliding window of recent prediction errors and, at every step, checks whether splitting the window into two sub-windows reveals a statistically significant difference in mean error rate [1]. It does this using the Hoeffding bound, a distribution-free concentration inequality that bounds false-positive and false-negative rates without assuming anything about the underlying data distribution. DDM (Gama et al., 2004) takes a different route, tracking the running error rate and its standard deviation, and signals drift once the current estimate climbs statistically far above the lowest rate seen since the last reset [2]. EDDM (Baena-García et al., 2006) instead tracks the mean distance between consecutive errors rather than the raw error rate, which makes it more sensitive to slow, gradual drift [3]. Hu et al. (2025) proposed DWOIDS, which uses two windows of different sizes to tell genuine drift apart from short-term noise, and benchmark against an Adaptive Random Forest with ADWIN, reporting F1 of 0.9210 on NSL-KDD and 0.9532 on CIC-IDS2017 for that baseline [12].

C. Adaptive and online learning for drift in IoT IDS

Abdel Wahab (2022) is the closest methodological comparator found in the literature. It is an online deep neural network for IoT IDS that detects both data and concept drift through PCA-based variance monitoring and an online outlier detector, and adapts by updating a Hedge-weighted, variable-depth DNN after every single sample [7]. It is evaluated on the DS2OS dataset (357,000 records, $\sim$6 classes) using 10-fold cross-validation; Section VII-B contrasts this against the present system in detail. Mulinka et al. (2019) show, empirically, that models updated continuously from live streams beat static models over extended operation on real network security data [8]. Yang and Shami (2021) propose a lightweight framework, LightGBM paired with an Optimised Adaptive Sliding Windowing method, and show that *triggered* retraining matches continuous online learning at a fraction of the compute cost: 99.92%/98.31% accuracy on IoTID20/NSL-KDD against an 84.25% offline baseline, at 7.8–9.1 ms/sample edge latency on a Raspberry Pi 3, though only for binary anomaly detection [11]. Amin et al. (2023) study drift in Industrial IoT sensor streams and point out that drift can arise from perfectly benign causes too, like device ageing or firmware updates, not only from new attacks [9]. Chu et al. (2024) propose drift *localisation*, identifying which features actually shifted rather than just detecting that something changed, and show that without adaptation, accuracy on Edge-IIoTset falls from 93.61% to 56.80%, recovering to an average of 93.93% once adaptation is applied [10].

D. Multi-detector comparisons and taxonomy surveys

Alqahtani (2024) is the only paper found that directly compares multiple error-rate drift detectors (ADWIN, DDM, EDDM, HDDM_A, HDDM_W, Page-Hinkley, KSWIN) on the same task, finding all seven within $\sim$0.3 F1 points of one another (0.9835–0.9872) on DAPT2020, which the author describes as "negligible variation" [13]. Carnier et al. (2025) pair ADWIN-triggered retraining with an Adaptive Random Forest across mixed IoT streams (Edge-IIoTset, MQTTSet, MQTT-IoT-IDS2020), reporting F1 of 0.988 ± 0.006, and show that a non-adaptive batch Random Forest collapses on those same heterogeneous streams [14]. Shyaa et al. (2024) survey ML/DL approaches to IDS and conclude that concept drift and feature drift are both largely ignored in existing work [15]. Salman and Abdulrahman (2023) lay out a formal taxonomy of drift types – sudden, gradual, incremental, and recurring – and note that most detectors, ADWIN included, are tuned for sudden drift and tend to struggle with gradual change [16].

E. The gap

- Liu (2025) has the right problem domain and a strong architecture, but no drift handling whatsoever [6].
- Abdel Wahab (2022) handles drift directly, but at a computational cost – continuous per-sample DNN updates – that is hard to justify on constrained IoT devices, and on a task with only $\sim$6 classes evaluated via cross-validation rather than a genuine temporal stream [7].

- Yang and Shami (2021) achieve lightweight, triggered adaptation, but only for binary anomaly detection [11].
- Alqahtani (2024) compares multiple detectors, but on binary APT detection, and doesn't report reaction latency [13].
- No study combines a practical, formally-grounded drift detector, an explicit fix for catastrophic forgetting, a full 33-class multi-attack IoT dataset, and a rigorous static-vs-adaptive comparison with a multi-detector reaction-latency analysis, all together. That combination is the gap this project addresses.

III. Dataset and Notation

A. CICIoT2023

CICIoT2023 [5], produced by the Canadian Institute for Cybersecurity, contains network traffic captured from a real IoT testbed. It comes as 63 CSV files (Merged01–Merged63), each holding approximately 700,000 rows, with 39 network-flow features and a `Label` column with the exact number varying between files. The experiments in this project use different subsets of these files, as described in Section III-B. The dataset spans 34 classes in total: benign traffic plus 33 attack types spread across six categories – DDoS (12 sub-types, e.g. ICMP/UDP/TCP/SYN Flood, fragmentation variants, Slowloris), DoS (4 sub-types), Mirai botnet (3 sub-types), Reconnaissance (5 sub-types), Web attacks (5 sub-types, e.g. XSS, SQL Injection, Command Injection), and MITM/Spoofing (3 sub-types). That is considerably more classes and diversity than commonly used alternatives such as NSL-KDD or CIC-IDS2017 offer, and far more than the ∼6-class DS2OS dataset used by the closest related work [7].

B. Notation

Three evaluation configurations are used throughout. *Approach 1* trains and tests on a single file (Merged01.csv, 712,297 rows) with an internal 80/20 split. *Approach 2* trains and tests on a stratified 10-file sample (2,000 common rows/file) with an internal split. *Approach 3* trains on a pooled sample from ten files (Merged01–10, 106,333 rows) and tests on a completely unseen file (Merged11.csv, 909,744 rows) – the only configuration that counts as a genuine generalisation test rather than a within-file split.

For the drift experiments, a synthetic three-phase stream is built from CICIoT2023's attack-type groupings: *Phase 1* (DDoS plus benign, used for initial training and streaming), *Phase 2* (DoS, Reconnaissance, and Mirai plus benign, stream-only), and *Phase 3* (Web/application attacks plus benign, stream-only, and the hardest of the three). Each phase draws 15,000 samples, split 85/15 into a streaming portion and a held-out portion that is never touched during training, streaming, or retraining, kept aside for a leak-free generalisation test. Data moves through the pipeline in chunks of 500 rows (`CHUNK_SIZE`=500); when a retrain is triggered, it uses a sliding window of the ∼1,500 most recent stream samples mixed with rehearsal samples drawn from a long-term reservoir (Section IV-D).

IV. Detection and Adaptation Methods

A. Static Random Forest baseline

A Random Forest classifier (50 estimators, `class_weight=balanced`) serves as the base learner throughout, in line with prior surveys that consistently rank Random Forest as a strong performer on IoT IDS datasets. For the baseline generalisation experiment, the static Random Forest is trained once on the pooled training sample from Merged01–10 and then evaluated on the completely unseen Merged11 without retraining. The static baseline acts as the control condition against which adaptation is measured.

B. ADWIN-triggered retraining

The adaptive model uses the same Random Forest architecture, but wraps ADWIN around the live prediction-error stream (1 for misclassified, 0 for correct). Once ADWIN's Hoeffding-bound test flags a significant change in mean error rate, the older sub-window is dropped and a retrain fires on the most recent samples mixed with rehearsal data (Section IV-D); ADWIN then resets itself.

C. DDM and EDDM as alternative detectors

To check whether ADWIN's formal-guarantee approach is actually necessary, or whether a simpler heuristic would do just as well, DDM and EDDM were hand-coded from their original papers – not pulled from a library – and dropped into an otherwise identical pipeline. During this work, both hand-coded detectors turned out to report zero drift detections. The reason was that both called an internal `reset()` immediately after setting `drift_detected = True`, and `reset()` unconditionally cleared that flag before the outer streaming loop ever got to read it. The fix was to remove that internal reset call, which brings DDM and EDDM in line with how ADWIN already worked (the outer loop calls `reset()` itself, after it has acted on the detected drift). The fix was verified against a synthetic error-rate-jump test before being rerun on real data.

D. Catastrophic-forgetting mitigation via a rehearsal buffer

An earlier version of the system, one that retrained only on the most recent post-drift samples, ran straight into catastrophic forgetting: Phase-1 held-out macro F1 collapsed to ∼0.01 once the model retrained on Phase 2 data, simply because nothing in the retrain buffer preserved the Phase-1 decision boundary. This was fixed with a long-term rehearsal reservoir, implemented through reservoir sampling (Algorithm R) [4], which keeps a fixed-capacity, uniformly-sampled memory of the entire stream seen so far and is never cleared on drift. Each retrain mixes ∼1,500 new post-drift samples with 1,000 samples drawn from the reservoir. Capacity and rehearsal count were tuned through a one-factor-at-a-time sweep (Table IV); a capacity of 6,000 with 1,000 rehearsal samples per retrain was adopted as the best balance between stability and plasticity.

E. Two-tier evaluation methodology

Every configuration is evaluated two ways. The first is *streaming (per-chunk)* accuracy/F1/FPR, comparing the static and adaptive models chunk by chunk as the stream progresses; this shows the adaptation dynamics clearly but can look a little optimistic, since the chunks right after a retrain are evaluated on samples related to what the model was just trained on. The second is *held-out generalisation*: evaluating the final static and adaptive models on data from each phase that was never streamed, trained on, or used in any retrain. This is the methodologically stronger test, and it plays the same role here that Approach 3 plays for the static baselines.

V. Experimental Setup

A. Data pipeline

Raw CSV rows are label-encoded, cleaned of NaN/inf values, and reduced to the 39 numeric feature columns. For the drift experiments, phase-specific attack-type subsets are sampled per phase (Section III-B) and split into stream and held-out partitions, stratified by class, with a minimum-class-size guard to keep very rare attack types from producing degenerate held-out splits.

B. Metrics

In keeping with the project's stated evaluation plan, results are reported using overall accuracy, balanced accuracy, macro-averaged F1 (to handle the 33-class imbalance), weighted F1, false positive rate (FPR, meaning benign traffic misclassified as an attack), the number of drift events detected, and total/mean retraining time. Full precision/recall/F1 classification reports and row-normalised confusion matrices are additionally produced for every configuration (Section VI-D).

C. Reproducibility

Random seeds are fixed (`random_state=42` for every Random Forest instance; `numpy.random.RandomState(42)` for reservoir sampling), so stream construction, phase ordering, and retrain triggers all come out deterministic across runs. All notebooks, results CSVs, and generated figures referenced in this paper are stored alongside the project code.

VI. Results and Analysis

A. Baseline detection results

Table I summarises the three static baselines. Only Approach 3 is a genuine generalisation test; Approaches 1–2 evaluate on a split of the same file(s) used for training.

Approach 1's binary model reaches 99.20% accuracy, 91.10% balanced accuracy, and an FPR of 17.41%; its multi-class model reaches 76.06% accuracy, macro F1 0.5916, weighted F1 0.7561, and a macro-averaged FPR of 0.76%. Approach 3's binary model reaches 99.04% accuracy and 94.10% balanced accuracy; its multi-class model reaches 75.43% accuracy, macro F1 0.5513, and weighted F1 0.7537. The gap in balanced accuracy between Approach 1 and Approach 3 is modest, which is a reassuring sign for the multi-class task specifically; Approach 2 lags well behind both (61.18%), consistent with its much smaller and less balanced internal test split. FPR was also measured for all three approaches. Approach 3 is used as the principal baseline generalisation configuration because Merged11 is completely unseen during training.

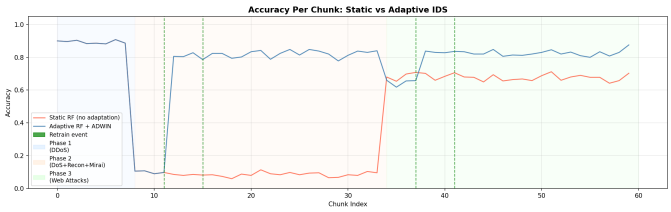

Fig. 1. Streaming accuracy, static vs. adaptive (ADWIN) Random Forest, across the three synthetic drift phases (shaded regions; dashed vertical lines mark retrain events).

B. Adaptive IDS under simulated drift

Fig. 1 shows binary-style streaming accuracy for the static and adaptive models across the three-phase stream. The static model, trained only on Phase 1 and never updated afterward, collapses from ∼0.90 to ∼0.08 at the Phase 1→2 boundary and stays collapsed through Phase 3. The adaptive model, by contrast, climbs back to ∼0.80 within a few chunks of each phase transition.

Averaged across the whole stream, mean accuracy rises from 45.09% (static) to 77.26% (adaptive), a gain of 32.17 percentage points, and mean macro F1 rises from 0.191 to 0.473. That gain does come at a cost: mean streaming FPR rises from 0.09% (static) to 3.41% (adaptive). In other words, adaptation trades a small, honestly-reported increase in false alarms for a much larger gain in catching the drifted attack types. ADWIN triggered four retrains in total across the stream – more than the two phase boundaries alone would suggest, since it can re-signal while a phase is still settling – at a total retraining cost of 0.21 seconds (0.053 s per retrain on average), cheap enough to run on every confirmed drift.

Table II breaks per-chunk macro F1 down by phase, and Table III reports the more demanding held-out generalisation test.

The adaptive model wins by a wide margin on Phase 2 and Phase 3 – the two phases the static model never saw – while giving up only a small amount of accuracy on Phase 1 (which the static model was trained on, and which the adaptive model has partly, but not catastrophically, forgotten).

C. Fixing catastrophic forgetting

Before the rehearsal buffer was added, Phase-1 held-out macro F1 collapsed to ∼0.01 as soon as the model retrained on Phase 2 data, because the retrain buffer held only new-phase samples. Table IV shows the sweep that produced the fix: a capacity of 6,000 with 1,000 rehearsal samples per retrain was adopted, which recovers Phase-1 held-out macro F1 to 0.67–0.69 while still gaining strongly on Phases 2–3 (Table III).

TABLE I
STATIC BASELINE RESULTS ACROSS THREE DATA STRATEGIES.

Approach	Train/Test Setup	Accuracy	Balanced Accuracy	Macro F1	FPR
1. Single-file	Merged01.csv, 80/20 split	99.20%	91.10%	0.5916	17.41%
2. 10-file sample	2,000 rows/file, internal split	98.42%	61.18%	0.5418	77.42%
3. Pooled + held-out	Train 106,333 rows → test 909,744 unseen rows	99.04%	94.10%	0.5513	11.09%

TABLE II
PER-CHUNK STREAMING MACRO F1 BY PHASE.

Phase	Static	Adaptive	Δ
Phase 1 (DDoS)	0.870	0.870	+0.000
Phase 2 (DoS+Recon+Mirai)	0.064	0.561	+0.497
Phase 3 (Web Attacks)	0.108	0.262	+0.154
Overall mean	0.191	0.473	+0.282

TABLE III
HELD-OUT (LEAK-FREE) MACRO F1 AND FPR BY PHASE, STATIC VS. ADAPTIVE.

Phase	F1 (S)	F1 (A)	FPR (S)	FPR (A)
Phase 1 (DDoS)	0.80	0.67	0.00%	4.11%
Phase 2 (DoS+Recon+Mirai)	0.06	0.49	0.00%	1.02%
Phase 3 (Web Attacks)	0.09	0.29	0.20%	3.08%

TABLE IV
REHEARSAL-BUFFER TUNING SWEEP (HELD-OUT MACRO F1, P1/P2/P3).

Config	Cap.	Rehearsal	Held-out F1
Capacity = 1500	1500	1000	0.648 / 0.512 / 0.309
Capacity = 6000 (adopted)	6000	1000	0.691 / 0.544 / 0.306
Rehearsal = 500	3000	500	0.627 / 0.499 / 0.309
Rehearsal = 2000	3000	2000	0.684 / 0.507 / 0.296

The pattern here is a textbook stability/plasticity tradeoff. Smaller rehearsal counts adapt fastest to new phases but hold on to old ones worst; going from a rehearsal count of 500 to 2,000 (both at capacity 3,000) improved Phase 1 retention (0.627→0.684), and increasing capacity to 6,000 pushed Phase 1 up further to 0.691, which is why it was adopted as the final configuration.

TABLE V
DETECTOR COMPARISON: SIGNALS, REACTION LATENCY, FPR, RETRAIN COST.

Detector	Signals	Chunks (P2/P3)	FPR	Retrain time
ADWIN	4	3 / 3	3.41%	0.29 s
DDM	2	3 / 8	2.75%	0.16 s
EDDM	5	3 / 12	3.75%	0.34 s

D. Full precision, recall, F1, accuracy, and confusion matrices

For Approach 1's binary model: Normal precision/recall/F1 = 0.83/0.83/0.83, Attack = 1.00/1.00/1.00, macro avg = 0.91/0.91/0.91, weighted avg = 0.99/0.99/0.99, accuracy 99%. For the multi-class model: accuracy 76.06%, balanced accuracy 59.54%, macro F1 0.5916, weighted F1 0.7561. Full per-class (33-class) precision/recall/F1 tables and row-normalised confusion-matrix heatmaps for every configuration – the Approach 1 and Approach 3 baselines, plus per-phase static-vs-adaptive pairs for the main ADWIN pipeline – are generated by the accompanying notebooks and stored under results/ as PNG figures. They are left out here to save space and summarised visually instead in Fig. 2, which compares all three detectors against the static baseline on the combined three-phase held-out set (33 classes, row-normalised).

E. ADWIN vs. DDM vs. EDDM

Fig. 3 plots per-chunk macro F1 for all three detectors against the static baseline. Table V reports drift-signal counts, reaction latency, mean FPR, and total retraining time.

All three detectors land within ∼0.05 macro F1 of one another on final combined-holdout accuracy (Fig. 2 caption), which lines up with Alqahtani's (2024) finding of "negligible variation" across seven detectors on DAPT2020 [13]. Where they differ is reaction speed. ADWIN reaches its first post-boundary retrain within 3 chunks of *both* phase transitions; DDM matches that on the first transition but takes 8 chunks on the second; EDDM takes 12 chunks despite firing the most drift signals overall (5, against ADWIN's 4 and DDM's 2). That reaction-latency angle isn't something the closest comparable published comparison reports [13]; it evaluates detector choice purely on aggregate F1.

F. Comparison against the wider literature

Table VI places this project's headline results next to the six most closely related published results found during the

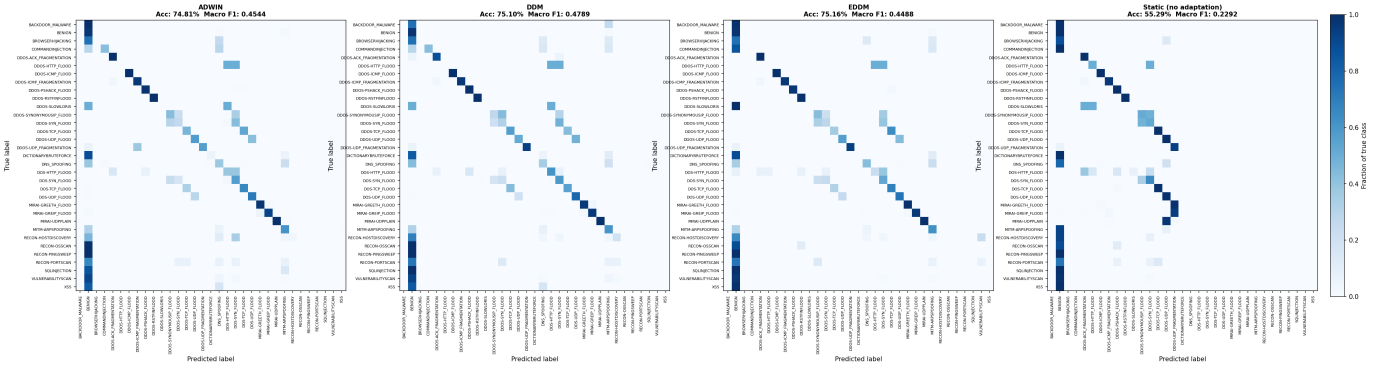

Fig. 2. Confusion matrices (row-normalised) on the combined 3-phase held-out set, 33 classes: ADWIN (Acc 74.81%, macro F1 0.4544), DDM (75.10%, 0.4789), EDDM (75.16%, 0.4488), and the static baseline (55.29%, 0.2292). The static baseline's diagonal visibly breaks down for Phase 2/3 classes it never trained on; all three adaptive detectors retain a clean diagonal across all phases.

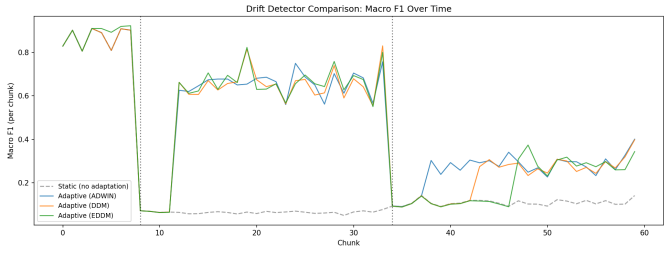

Fig. 3. Macro F1 over time, ADWIN vs. DDM vs. EDDM vs. static baseline, across the three-phase stream.

literature search. Most prior work evaluates binary classification, and this project's full 33-way multi-class task is a harder problem, so these numbers are shown for context rather than as a like-for-like ranking.

No paper found evaluates full multi-class drift adaptation on CICIoT2023, and none report reaction latency alongside a multi-detector comparison the way this project's DDM/EDDM benchmark does.

VII. Discussion

A. Positioning against the base paper, Liu (2025)

Table VII places this project's accuracy and macro F1 next to Liu's reported ToN-IoT results, for context rather than as a controlled benchmark: the datasets (CICIoT2023 vs. ToN-IoT), the task granularity, and the architectures (a single Random Forest plus ADWIN vs. a three-layer Autoencoder→XGBoost→RNN-BiLSTM pipeline) are all different.

What this comparison does show cleanly is a result internal to this project: under simulated concept drift, adaptation recovers ∼20 percentage points of accuracy and roughly doubles macro F1 compared to doing nothing (55.29%→75.2%, 0.229→0.479). That is a failure mode Liu's static architecture has no way to detect or correct, since it was never evaluated under drift in the first place. Liu also reports edge-deployment metrics (latency, throughput, memory, CPU) that this project has not measured on physical hardware, discussed as a limitation in Section VII-C.

B. Comparison against the closest related work, Abdel Wahab (2022)

Abdel Wahab (2022) is the paper most directly comparable in problem framing here – IoT intrusion detection under concept and data drift – but it differs from this system on almost every design decision that matters. On *dataset and task*: DS2OS with ∼6 classes, against CICIoT2023's 34 classes here. On *drift detection*: PCA-based distributional variance monitoring plus an online outlier detector, which is a heuristic check with no formal statistical guarantee, against ADWIN's Hoeffding-bound test (and the DDM/EDDM comparison in Section VI-E), both of which come with known false-positive/negative bounds. On *adaptation cost*: a DNN updated after *every* single sample through Hedge Backpropagation across 15 dynamically weighted hidden layers. No runtime figure is reported in that paper, but continuous per-sample deep-network updates are architecturally heavier than the four retrains used here, each taking 0.05–0.34 s. On *catastrophic forgetting* – where the two approaches disagree most – Abdel Wahab explicitly argues against preserving old knowledge, stating that online learning is preferable precisely because it "imposes no limitations on the model in terms of not forgetting previous knowledge," on the reasoning that old attack-label relationships may no longer hold. This project takes the opposite view: it treats forgetting as a measured failure mode (Section VI-C) and fixes it, because in the scenario evaluated here, earlier attack types (DDoS in Phase 1, for instance) stay valid threats even after new attack types show up, unlike Abdel Wahab's implicit assumption that old knowledge is safe to throw away. On *evaluation methodology*: 10-fold cross-validation, which does not enforce any temporal ordering, against this project's explicit three-phase temporal stream with leak-free held-out testing per phase. And on *metrics*: accuracy, precision, recall, and F1 only, against this project's additional FPR, drift-event count, and retraining-time reporting, matching what the project proposal actually specified.

C. Threats to validity

A few limitations are worth naming honestly. First, evaluation covers only CICIoT2023; whether the results generalise

TABLE VI
COMPARISON AGAINST THE REVIEWED CONCEPT-DRIFT IDS LITERATURE.

Study	Dataset / Task	Approach	Key Result
Yang & Shami (2021) [11]	IoTID20 / NSL-KDD, binary	LightGBM + OASW, triggered	99.92% / 98.31% acc. vs. 84.25% offline; edge latency 7.8–9.1 ms/sample (RPi3)
Chu et al. (2024) [10]	Edge-IIoTset, multi-concept	Drift localisation + adaptive XGBoost	Accuracy 93.61%→56.80% without adaptation; recovers to 93.93% avg. with adaptation
Alqahtani (2024) [13]	DAPT2020, binary (SDN)	7 detectors incl. ADWIN/DDM/EDDM	All 7 within ∼0.3 F1 points (0.9835–0.9872) – "negligible variation", no latency reported
Hu et al., DWOIDS (2025) [12]	NSL-KDD / CIC-IDS2017, binary	Dual-window + Hoeffding tree; ARF+ADWIN baseline	ARF+ADWIN: F1 0.9210 / 0.9532; DWOIDS: F1 0.9364 / 0.9653
Carnier et al. (2025) [14]	Mixed IoT streams, binary	ADWIN + Adaptive Random Forest	ARF: F1 0.988±0.006; batch RF collapses under drift on the same streams
This work	CICIoT2023, 33-class, multi-concept	RF + ADWIN/DDM/EDDM, rehearsal buffer	Combined holdout Acc. 74.8–75.2%, macro F1 0.449–0.479 (adaptive) vs. 55.29%/0.229 (static)

TABLE VII
COMMON METRICS VS. LIU (2025) – CONTEXT ONLY, NOT A CONTROLLED BENCHMARK.

System	Task	Acc.	Macro F1
This work – static, no drift	CICIoT2023, 33-class	76.06%	0.5916
This work – static, under drift	CICIoT2023, drift	55.29%	0.2292
This work – adaptive, under drift	CICIoT2023, drift	74.8–75.2%	0.449–0.479
Liu (2025) – ToN-IoT DS1 [6]	ToN-IoT, static	95%	0.94
Liu (2025) – ToN-IoT DS2 [6]	ToN-IoT, static	92%	0.91

to other IoT traffic distributions (Bot-IoT, ToN-IoT, Edge-IIoTset) is untested. Second, the three-phase drift sequence is built from CICIoT2023's own attack-type groupings rather than captured from a live deployment evolving over time, so it is a controlled proxy for drift rather than something directly observed. Third, resource and latency metrics, which both Liu (2025) and Yang and Shami (2021) report for edge deployment, have not been measured on physical hardware here; that is a deliberate scope decision (physical edge-device evaluation was outside the final evaluation scope), not an oversight. Fourth, only Random Forest was evaluated as the base learner, so how other model families would behave under the same drift-adaptive wrapper is still an open question.

VIII. CONCLUSION

This paper presented an Adaptive Concept Drift-Aware IDS that pairs a Random Forest classifier with ADWIN-triggered retraining and a tuned rehearsal buffer, evaluated on the full 33-attack-type CICIoT2023 taxonomy under a synthetic three-phase drift stream. Adaptation lifts mean streaming accuracy from 45.09% to 77.26% (+32.17 percentage points) and macro F1 from 0.191 to 0.473, at a total retraining cost of 0.21 seconds across four retrains. The rehearsal buffer recovers Phase-1 held-out macro F1 from an uncontrolled-forgetting collapse of ∼0.01 to a tuned 0.67–0.69. Benchmarked against hand-coded DDM and EDDM, all three detectors land on similar final accuracy, but ADWIN reacts fastest and most consistently to both simulated phase transitions – a reaction-latency angle the closest comparable published comparison doesn't report. Relative to the project's base paper, Liu (2025), a static architecture with no drift handling, adaptation recovers roughly 20 accuracy points and doubles macro F1 under drift: a scenario Liu's design simply has no way to detect or correct. Relative to the closest methodological comparator, Abdel Wahab (2022), this system uses a formally-grounded, far cheaper triggered-retraining mechanism, and unlike that work, treats catastrophic forgetting as something to measure and fix rather than an acceptable side effect of adapting. Future work includes testing generalisation across other datasets, measuring resource and latency numbers on real edge hardware, and trying the same drift-adaptive wrapper around other base classifiers.

USE OF AI

While preparing this report, I used Claude (Anthropic) as an assistant for research and writing. This was especially helpful for deepening my understanding of the concepts covered in the report — such as concept drift detection (ADWIN, DDM, EDDM), catastrophic forgetting and rehearsal-based mitigation, and the structure and attack taxonomy of the CICIoT2023 dataset — by having Claude provide clarifying explanations of these concepts and relate them to what I had already read in the literature, including the base paper (Liu, 2025) and the closest related work (Abdel Wahab, 2022). It was also used to draft initial text for some sections, suggest improvements to sentence structure, help hand-code and debug the DDM and EDDM drift detectors from their

original papers, add evaluation code for metrics I had not originally implemented (false positive rate, retraining time, full precision/recall/F1 reporting), and resolve LaTeX layout issues encountered during compilation in Overleaf. The selection of papers reviewed, the identification of the research gap relative to Liu (2025) and Abdel Wahab (2022), the choice of the CICIoT2023 benchmark, and the design of the three-phase drift stream and static-vs-adaptive experimental comparison are entirely my own work. Every claim about a referenced paper was checked against the original source before inclusion in this document. This declaration is made in accordance with the capstone project's policy on the use of AI tools.

ACKNOWLEDGMENT

I would like to thank my supervisor, Dr. Waqar Shahid Qureshi, for his guidance and feedback throughout this capstone project, from the initial proposal through to this final report.

REFERENCES

[1] A. Bifet and R. Gavalda, "Learning from Time-Changing Data with Adaptive Windowing," in *Proc. 7th SIAM Int. Conf. Data Mining (SDM)*, Minneapolis, MN, 2007, pp. 443–448, doi: 10.1137/1.9781611972771.42.

[2] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with Drift Detection," in *Advances in Artificial Intelligence – SBIA 2004*, Lecture Notes in Computer Science, vol. 3171, Springer, 2004, pp. 286–295.

[3] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early Drift Detection Method," in *Proc. 4th ECML PKDD Int. Workshop on Knowledge Discovery from Data Streams*, 2006, pp. 77–86.

[4] J. S. Vitter, "Random Sampling with a Reservoir," *ACM Transactions on Mathematical Software*, vol. 11, no. 1, pp. 37–57, 1985, doi: 10.1145/3147.3165.

[5] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023, doi: 10.3390/s23135941.

[6] B. Liu, "A Three-Layer Hierarchical Intrusion Detection System for IoT Networks," MSc thesis, University of Galway, Supervisor: Dr. Waqar Shahid Qureshi, Aug. 2025.

[7] O. Abdel Wahab, "Intrusion Detection in the IoT Under Data and Concept Drifts: Online Deep Learning Approach," *IEEE Internet of Things Journal*, vol. 9, no. 20, pp. 19706–19716, Oct. 2022, doi: 10.1109/JIOT.2022.3167005.

[8] P. Mulinka, P. Casas, and J. Vanerio, "Continuous and Adaptive Learning over Big Streaming Data for Network Security," in *Proc. IEEE 8th Int. Conf. Cloud Networking (CloudNet)*, Coimbra, Portugal, 2019, pp. 1–4, doi: 10.1109/CloudNet47604.2019.9064134.

[9] M. Amin, F. Al-Obeidat, A. Tubaishat, B. Shah, S. Anwar, and T. A. Tanveer, "Cyber Security and Beyond: Detecting Malware and Concept Drift in AI-Based Sensor Data Streams Using Statistical Techniques," *Computers and Electrical Engineering*, vol. 108, p. 108702, 2023, doi: 10.1016/j.compeleceng.2023.108702.

[10] R. Chu, P. Jin, H. Qiao, and Q. Feng, "Intrusion Detection in the IoT Data Streams Using Concept Drift Localization," *AIMS Mathematics*, vol. 9, no. 1, pp. 1535–1561, 2024, doi: 10.3934/math.2024076.

[11] L. Yang and A. Shami, "A Lightweight Concept Drift Detection and Adaptation Framework for IoT Data Streams," *IEEE Internet of Things Magazine*, vol. 4, no. 2, pp. 96–101, Jun. 2021, doi: 10.1109/IOTM.0001.2000260.

[12] X. Hu, D. Ma, W. Wang, and F. Liu, "Dual Adaptive Windows Toward Concept-Drift in Online Network Intrusion Detection," in *Lecture Notes in Computer Science (ICCS 2025)*, vol. 15903, Springer, 2025, doi: 10.1007/978-3-031-97626-1_15.

[13] A. H. Alqahtani, "An Incremental Hybrid Adaptive Network-Based IDS in SDN to Detect Stealth Attacks," arXiv preprint arXiv:2404.01109, 2024.

[14] R. M. Carnier, L. Lahesoo, and K. Fukuda, "Binary Anomaly Detection in Streaming IoT Traffic under Concept Drift," arXiv preprint arXiv:2510.27304, 2025.

[15] M. A. Shyaa, N. F. Ibrahim, Z. Zainol, R. Abdullah, M. Anbar, and L. Alzubaidi, "Evolving Cybersecurity Frontiers: A Comprehensive Survey on Concept Drift and Feature Dynamics Aware Machine and Deep Learning in Intrusion Detection Systems," *Engineering Applications of Artificial Intelligence*, vol. 137, p. 109143, 2024, doi: 10.1016/j.engappai.2024.109143.

[16] N. S. Salman and A. A. Abdulrahman, "Survey on Intrusion Detection System Based on Analysis Concept Drift: Status and Future Directions," *Int. J. Nonlinear Anal. Appl.*, vol. 14, no. 1, 2023.